

Guía Técnica: Dragones AR — Análisis del Código

. Estructura General del Proyecto

El archivo `index.html` es una aplicación de **Realidad Aumentada (AR)** que utiliza:

- **Three.js** (biblioteca 3D)
- **WebXR** (API de realidad virtual/aumentada del navegador)
- **GLTFLoader** (cargador de modelos 3D en formato GLB/GLTF)

2. Configuración Inicial

2.1 Import Map (líneas 33-41)

```
<script type="importmap">
{
  "imports": {
    "three": "./build/three.module.js",
    "three/addons/": "./jsm/"
  }
}
</script>
```

Función: Le dice al navegador dónde encontrar los módulos de Three.js. La palabra `"three"` se resuelve a `./build/three.module.js`, y `"three/addons/"` apunta a `./jsm/` (contiene utilidades como `ARButton`, `GLTFLoader`, etc.).

2.2 Constantes de Configuración (líneas 54-56)

```
const ESCALA_DRAGON = 1.0;
const ALTURA_DRAGON = 0.0;
```

Constante	Propósito
<code>`ESCALA_DRAGON`</code>	Factor de escala aplicado al modelo del dragón. El valor <code>`1.0`</code> asume que el modelo ya fue escalado en Unity (a 0.1) antes de exportar.
<code>`ALTURA_DRAGON`</code>	Posición vertical (eje Y) del dragón en el espacio AR. <code>`0.0`</code> lo coloca al mismo nivel que la nave.

2.3 Array TARGETS (líneas 59-62)

```
const TARGETS = [  
  { nombre: 'Dragon Azul', tarjeta: './images/tarjeta.png', modelo: './models/Blue.glb', animacion: 4 },  
  { nombre: 'Dragon Rojo', tarjeta: './images/tarjeta2.png', modelo: './models/Red.glb', animacion: 0 }  
];
```

Función: Define los datos de cada dragón disponible:

- **nombre:** Texto mostrado en la UI
- **tarjeta:** Imagen de la miniatura
- **modelo:** Ruta del archivo GLB
- **animacion:** **Índice numérico** de la animación dentro del archivo GLB (no el nombre)

Nota técnica: Se usa índice numérico porque los nombres de animación pueden variar según el exportador (FBX2glTF, Blender, Unity, etc.). El índice es invariante.

3. Funciones de la UI

3.1 cargarTarjetas() (líneas 74-93)

```
function cargarTarjetas() {  
  return Promise.all(TARGETS.map((t, i) => new Promise((resolve, reject) => {  
    const img = new Image();  
    img.crossOrigin = 'anonymous';  
    img.onload = () => { /* crear elementos DOM */ resolve(); });  
    img.onerror = () => reject(new Error('No se pudo cargar: ' + t.tarjeta));  
    img.src = t.tarjeta;  
  })));  
}
```

Función: Carga las imágenes de las tarjetas (miniaturas) de forma asíncrona. Para cada dragón:

1. Crea un objeto `Image` en JavaScript
2. Espera a que cargue (`onload`)
3. Crea los elementos HTML (div contenedor, imagen, etiqueta de texto)
4. Añade el evento `click` para seleccionar ese dragón

Retorna: Una `Promise` que se resuelve cuando TODAS las imágenes han cargado.

3.2 seleccionarDragon(index) (líneas 95-102)

```
function seleccionarDragon(index) {  
  selectedIndex = index;  
  document.querySelectorAll('.target').forEach((el, i) => {  
    el.classList.toggle('selected', i === index);  
  });  
  btnAR.disabled = false;  
  statusEl.textContent = 'Seleccionado: ' + TARGETS[index].nombre + '. Toca Iniciar AR.';  
  statusEl.className = 'ok';  
}
```

Función: Responde al evento `click` en una tarjeta:

- Guarda el índice del dragón seleccionado
- Aplica la clase CSS `selected` (borde azul) a la tarjeta activa
- **Habilita el botón "Iniciar AR"** (`btnAR.disabled = false`)
- Actualiza el mensaje de estado

3.3 cargarModelos() (líneas 104-113)

```
function cargarModelos() {  
  const loader = new GLTFLoader();  
  return Promise.all(TARGETS.map((t, i) => new Promise((resolve, reject) => {  
    loader.load(t.modelo,  
      (gltf) => resolve({ gltf, nombre: t.nombre, index: i, animacionBuscada: t.animacion }),  
      undefined,  
      (err) => reject(new Error('No se pudo cargar ' + t.modelo + ': ' + err.message))  
    });  
  })));  
}
```

Función: Precarga los modelos 3D (archivos .glb) usando GLTFLoader.

Parámetros de loader.load():

1.url: Ruta del archivo

2.onLoad: Callback cuando carga exitosamente. Resuelve la Promise con un objeto que contiene:

- gltf: El objeto GLTF completo (escena, animaciones, materiales)

- nombre, index, animacionBuscada: Metadatos del dragón

3.onProgress: No se usa (undefined)

4.onError: Callback de error, rechaza la Promise

Retorna: Promise que se resuelve cuando AMBOS modelos han cargado.

4. Función Principal del Modelo 3D

4.1 crearModelo(data) (líneas 115-140)

```
function crearModelo(data) {  
  const container = new THREE.Group();  
  const model = data.gltf.scene; // SIN .clone()  
  
  container.scale.set(ESCALA_DRAGON, ESCALA_DRAGON, ESCALA_DRAGON);  
  container.rotation.y = 0;  
  
  const box = new THREE.Box3().setFromObject(model);  
  const center = box.getCenter(new THREE.Vector3());  
  model.position.set(-center.x, -center.y, -center.z);  
  
  container.add(model);  
  
  let mixer = null;  
  if (data.gltf.animations && data.gltf.animations.length > 0) {  
    mixer = new THREE.AnimationMixer(model);  
    const indice = data.animacionBuscada;  
    const clip = data.gltf.animations[indice] || data.gltf.animations[0];  
    mixer.clipAction(clip).play();  
  }  
  
  return { container, mixer, model };  
}
```

Función: Prepara el modelo 3D para su uso en AR.

Paso a paso:

Paso	Código	Explicación
	<pre>`const container = new 1 THREE.Group()`</pre>	Crea un grupo vacío que servirá como "contenedor padre" **Usa el modelo directamente, sin clonar** . Esto es crítico para que las animaciones de esqueleto (skinned mesh) funcionen correctamente.
	<pre>2 `const model = data.gltf.scene` 3 `container.scale.set(...)` 4 `container.rotation.y = 0` `new   THREE.Box3().setFromObject(model`</pre>	Aplica escala uniforme al contenedor Rota el contenedor 0 radianes (mira hacia el usuario)
	<pre>5 l)`</pre>	Calcula la caja envolvente del modelo Centra el modelo en su origen local, compensando su centro geométrico
	<pre>6 `model.position.set(-center.x, ...)` 7 `container.add(model)` `new</pre>	Añade el modelo al contenedor
	<pre>8 THREE.AnimationMixer(model)` 9 `data.gltf.animations[indice]` 10 `mixer.clipAction(clip).play()`</pre>	Crea un mezclador de animaciones vinculado al modelo Obtiene el clip de animación por índice numérico Crea una acción de animación y la reproduce

Nota técnica crucial: El modelo **no se clona** (`data.gltf.scene` en vez de `data.gltf.scene.clone()`). En modelos con esqueleto (skinned mesh), `.clone()` rompe las referencias entre los huesos y la malla, haciendo que las animaciones no funcionen. Esta fue la solución clave del proyecto.

5. Inicialización de la Aplicación

5.1 `init()` (líneas 142-155)

```
async function init() {  
  try {  
    statusEl.textContent = 'Cargando tarjetas...';  
    await cargarTarjetas();  
    statusEl.textContent = 'Descargando modelos...';  
    modelosData = await cargarModelos();  
    statusEl.textContent = 'Listo. Selecciona un dragón.';  
    statusEl.className = 'info';  
  } catch (err) {  
    statusEl.textContent = 'Error: ' + err.message;  
    statusEl.className = 'err';  
    console.error(err);  
  }  
}  
init();
```

Función: Secuencia de inicialización asíncrona:

1. Carga las tarjetas (imágenes UI)
2. Precarga los modelos 3D
3. Muestra "Listo" cuando todo está cargado

Es **async/await** porque ambas operaciones son asíncronas (Promise-based).

6. Función de Inicio de AR

6.1 iniciarAR() (líneas 157-220)

```
function iniciarAR() {  
  if (selectedIndex < 0) { /* validación */ return; }  
  
  const data = modelosData[selectedIndex];  
  
  // DEBUG: muestra animación disponible  
  const primeraAnim = data.gltf.animations[0];  
  const nombrePrimera = primeraAnim ? primeraAnim.name : 'NINGUNA';  
  statusEl.innerHTML = `Anim[0]: "${nombrePrimera}"...<button  
id="btnContinuar">CONTINUAR</button>`;  
  
  document.getElementById('btnContinuar').addEventListener('click', () => {  
    document.getElementById('ui').style.display = 'none';  
  
    // --- INICIO DEL MUNDO 3D ---  
    clock = new THREE.Clock();  
    scene = new THREE.Scene();  
    camera = new THREE.PerspectiveCamera(70, window.innerWidth / window.innerHeight, 0.01, 20);  
  
    // Iluminación  
    scene.add(new THREE.HemisphereLight(0xa5a5a5, 0x898989, 3));  
    const light = new THREE.DirectionalLight(0xffffff, 3);  
    light.position.set(1, 1, 1).normalize();  
    scene.add(light);  
  
    // Carga de la nave  
    const loaderNave = new GLTFLoader();
```



```
loaderNave.load('./models/gltf/Nave.gltf', function (gltf) {  
  gltf.scene.scale.set(0.003, 0.003, 0.003);  
  gltf.scene.position.set(-0.8, 0.3, -1.5);  
  scene.add(gltf.scene);  
});  
  
// Carga del dragón  
const { container, mixer } = crearModelo(data);  
container.position.set(1.0, ALTURA_DRAGON, -1.5);  
scene.add(container);  
dragonContainer = container;  
dragonMixer = mixer;  
  
// Renderer WebGL  
renderer = new THREE.WebGLRenderer({ antialias: true, alpha: true });  
renderer.setPixelRatio(window.devicePixelRatio);  
renderer.setSize(window.innerWidth, window.innerHeight);  
renderer.xr.enabled = true;  
document.body.appendChild(renderer.domElement);  
  
// Botón AR nativo de WebXR  
document.body.appendChild(ARButton.createButton(renderer));  
  
window.addEventListener('resize', onWindowResize);  
renderer.setAnimationLoop(animate);  
});  
}
```

Función: Orquesta todo el inicio de la sesión AR.

Fases:

Fase	Descripción
Validación	Verifica que se haya seleccionado un dragón
Debug UI	Muestra información de la animación y espera confirmación del usuario
Entrada a AR	Al tocar "Continuar", oculta la UI y construye el mundo 3D
Escena	Crea <code>`THREE.Scene`</code> (contenedor de todos los objetos 3D)
Cámara	<code>`PerspectiveCamera`</code> con campo de visión 70°, rango 0.01 a 20 metros <code>`HemisphereLight`</code> (luz ambiental) + <code>`DirectionalLight`</code> (luz direccional tipo sol)
Luces	
Nave	Carga asíncrona del modelo GLTF de la nave, con escala y posición fijas
Dragón	Crea el modelo con <code>`crearModelo()`</code> , posiciona a la derecha de la nave <code>`WebGLRenderer`</code> con fondo transparente (<code>`alpha: true`</code>) para ver la cámara real detrás
Render	
WebXR	Activa <code>`renderer.xr.enabled = true`</code> y añade el botón AR nativo
Loop	<code>`setAnimationLoop(animate)`</code> inicia el ciclo de renderizado

7. Ciclo de Animación

7.1 `animate()` (líneas 230-234)

```
function animate() {  
  const delta = clock.getDelta();  
  if (dragonMixer) dragonMixer.update(delta);  
  renderer.render(scene, camera);  
}
```

unción: Se ejecuta en cada frame (60 veces por segundo típicamente).

Línea	Función
	Obtiene el tiempo transcurrido desde el último frame
<code>`clock.getDelta()`</code>	(en segundos)
	Avanza la animación del dragón según el tiempo transcurrido
<code>`dragonMixer.update(delta)`</code>	
<code>`renderer.render(scene, camera)`</code>	Dibuja la escena completa desde la perspectiva de la cámara

Nota: En WebXR se usa `setAnimationLoop()` en lugar de `requestAnimationFrame()`. Three.js maneja automáticamente la sincronización con el headset o el dispositivo móvil. **Nota:** En WebXR se usa `setAnimationLoop()` en lugar de `requestAnimationFrame()`. Three.js maneja automáticamente la sincronización con el headset o el dispositivo móvil.

8. Manejo de Eventos

8.1 onWindowResize() (líneas 222-226)

```
function onWindowResize() {  
  camera.aspect = window.innerWidth / window.innerHeight;  
  camera.updateProjectionMatrix();  
  renderer.setSize(window.innerWidth, window.innerHeight);  
}
```

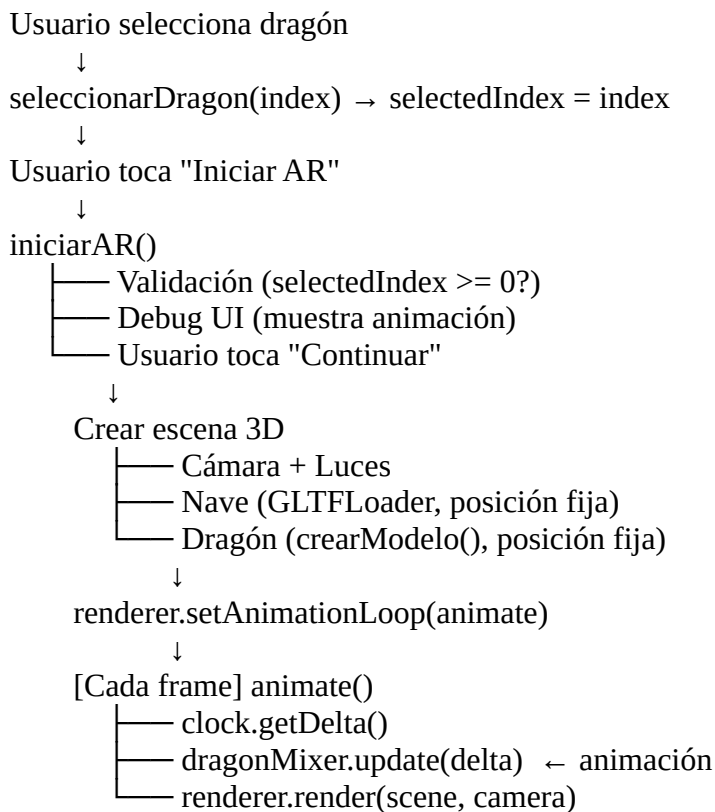
Función: Ajusta la cámara y el renderer cuando el usuario rota el teléfono (cambio de orientación portrait/landscape).

8.2 Evento del Botón AR (línea 236)

```
btnAR.addEventListener('click', iniciarAR);
```

Función: Vincula el botón "Iniciar AR" de la UI con la función `iniciarAR( )`.

9. Diagrama de Flujo de Datos



10. Lecciones Clave del Proyecto

Problema	Causa	Solución
Dragón gigante	Modelo exportado a escala 1.0 de Unity	Exportar a escala 0.1 en Unity, usar <code>`ESCALA_DRAGON = 1.0`</code>
Animación no funciona	<code>`clone()`</code> rompe referencias de huesos	Usar modelo directo: <code>`const model = data.gltf.scene`</code>
Nombres de animación inconsistentes	Exportadores cambian mayúsculas/minúsculas	Usar índices numéricos en lugar de nombres Ajustar <code>`ALTURA_DRAGON`</code> manualmente
Posición incorrecta	Coordenadas Y desfasadas	<code>`container.rotation.y = 0`</code>
Modelo de espaldas	Rotación por defecto	

Código QR

